

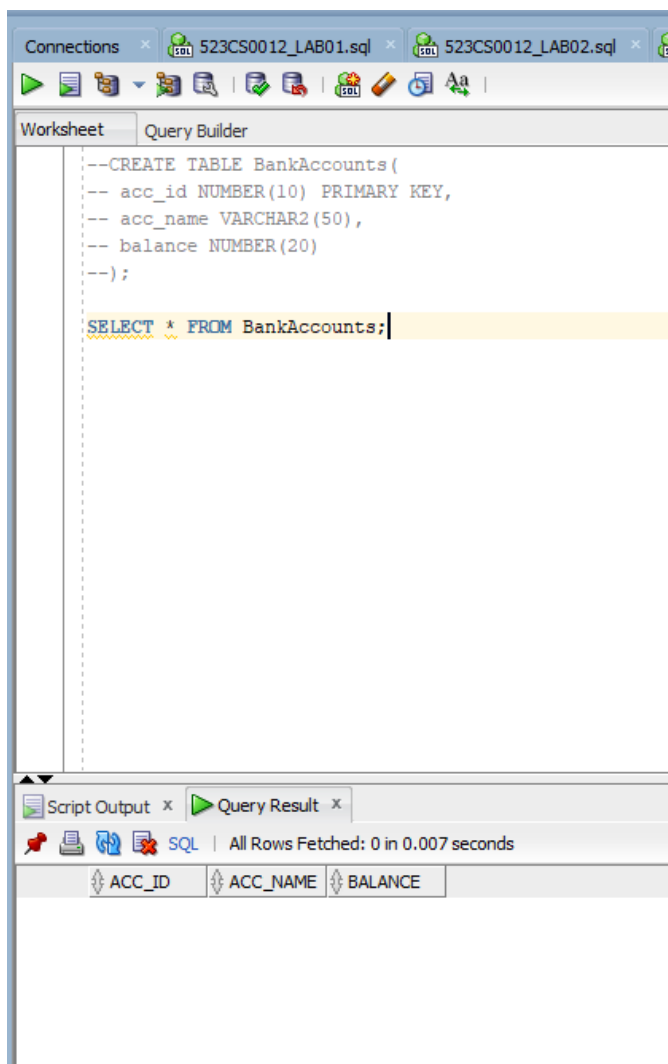
ASSIGNMENT – 04

TRANSACTION CONTROL COMMANDS

523cs0012 – GAURAV SINGH

Assignment-4.1 (BANKING SYSTEM)

BASIC COMMIT & ROLLBACK



Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sc

Worksheet Query Builder

```
--CREATE TABLE BankAccounts(  
-- acc_id NUMBER(10) PRIMARY KEY,  
-- acc_name VARCHAR2(50),  
-- balance NUMBER(20)  
--);  
  
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (101, 'Member_1', 25000);  
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (102, 'Member_2', 8000);  
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (103, 'Member_4', 12000);  
  
SELECT * FROM BankAccounts;
```

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.001 seconds

	ACC_ID	ACC_NAME	BALANCE
1	101	Member_1	25000
2	102	Member_2	8000
3	103	Member_4	12000

Connections

523CS0012_LAB01.sql

523CS0012_LAB02.sql

523CS0012_LAB03.sql

523CS0012_LAB04.sql



0.055 seconds

Worksheet

Query Builder

--CREATE TABLE BankAccounts(
-- acc_id NUMBER(10) PRIMARY KEY,
-- acc_name VARCHAR2(50),
-- balance NUMBER(20)
--);

--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (101, 'Member_1', 25000);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (102, 'Member_2', 8000);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (103, 'Member_4', 12000);

--UPDATE BankAccounts
--SET balance = balance + 2500
--WHERE acc_id = 101;

--

--SELECT * FROM BankAccounts;

Script Output

Query Result



Task completed in 0.055 seconds

1 row updated.

1 row updated.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	30000
102	Member_2	8000
103	Member_4	12000

1 row updated.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	27500
102	Member_2	8000
103	Member_4	12000

Connections

523CS0012_LAB01.sql

523CS0012_LAB02.sql

523CS0012_LAB03.sql

523CS0012_LAB04.sql



Worksheet

Query Builder

```
-- acc_name VARCHAR2(50),
-- balance NUMBER(20)
--);

--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (101, 'Member_1', 25000);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (102, 'Member_2', 8000);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (103, 'Member_4', 12000);

--UPDATE BankAccounts
--SET balance = balance + 2500
--WHERE acc_id = 101;

--
--COMMIT;

--UPDATE BankAccounts
--SET balance = balance - 1500
--WHERE acc_id = 102;

SELECT * FROM BankAccounts;
```

Script Output

Query Result



SQL

All Rows Fetched: 3 in 0.001 seconds

	ACC_ID	ACC_NAME	BALANCE
1	101	Member_1	27500
2	102	Member_2	6500
3	103	Member_4	12000

Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql

0.068 seconds

Worksheet Query Builder

```
-- balance NUMBER(20)
--);

--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (101, 'Member_1', 25000);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (102, 'Member_2', 8000);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (103, 'Member_4', 12000);

--UPDATE BankAccounts
--SET balance = balance + 2500
--WHERE acc_id = 101;

--
--COMMIT;

--UPDATE BankAccounts
--SET balance = balance - 1500
--WHERE acc_id = 102;

--ROLLBACK;

SELECT * FROM BankAccounts;
```

Script Output x Query Result x

Task completed in 0.068 seconds

ACC_ID	ACC_NAME	BALANCE
101	Member_1	27500
102	Member_2	8000
103	Member_4	12000

Commit complete.

1 row updated.

Rollback complete.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	27500
102	Member_2	8000
103	Member_4	12000

TRANSACTIONS WITH SAVEPOINTS & SERIALIZABLE

The screenshot displays the SQL Developer interface with a script editor and a query results window.

Script Editor:

```
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (103, 'Member_4', 12000);

--UPDATE BankAccounts
--SET balance = balance + 2500
--WHERE acc_id = 101;

--
--COMMIT;

--UPDATE BankAccounts
--SET balance = balance - 1500
--WHERE acc_id = 102;

--ROLLBACK;
--

--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (104, 'Member_5', 16200);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (105, 'Member_6', 21500);
SELECT * FROM BankAccounts;
```

Query Results:

Script Output x Query Result x

SQL | All Rows Fetched: 5 in 0.001 seconds

	ACC_ID	ACC_NAME	BALANCE
1	101	Member_1	27500
2	104	Member_5	16200
3	102	Member_2	8000
4	103	Member_4	12000
5	105	Member_6	21500



Worksheet Query Builder

```
--  
--COMMIT;  
  
--UPDATE BankAccounts  
--SET balance = balance - 1500  
--WHERE acc_id = 102;  
  
--ROLLBACK;  
--  
  
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (104, 'Member_5', 16200);  
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (105, 'Member_6', 21500);  
  
--UPDATE BankAccounts  
--SET balance = balance - 5000  
--WHERE acc_name = 'Member_1';  
  
SELECT * FROM BankAccounts;
```

Script Output x Query Result x

SQL | All Rows Fetched: 5 in 0.001 seconds

	ACC_ID	ACC_NAME	BALANCE
1	101	Member_1	22500
2	104	Member_5	16200
3	102	Member_2	8000
4	103	Member_4	12000
5	105	Member_6	21500

Connections

523CS0012_LAB01.sql

523CS0012_LAB02.sql

523CS0012_LAB03.sql

523CS0012_LAB04.sql



0.097 seconds

Worksheet

Query Builder

```
--SET balance = balance - 1500
--WHERE acc_id = 102;

--ROLLBACK;
--

--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (104, 'Member_5',16200);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (105, 'Member_6', 21500);

--UPDATE BankAccounts
--SET balance = balance - 5000
--WHERE acc_name = 'Member_1';

--SAVEPOINT save_point01;

--UPDATE BankAccounts
--SET balance = balance + 5000
--WHERE acc_name = 'Member_2';

SELECT * FROM BankAccounts;
```

Script Output

Query Result



Task completed in 0.097 seconds

1 row inserted.

1 row updated.

Savepoint created.

1 row updated.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	16200
102	Member_2	13000
103	Member_4	12000
105	Member_6	21500

Connections | 523CS0012_LAB01.sql | 523CS0012_LAB02.sql | 523CS0012_LAB03.sql | 523CS0012_LAB04.sql

0.029 seconds

Worksheet | Query Builder

```
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (104, 'Member_5',16200);
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (105, 'Member_6', 21500);

--UPDATE BankAccounts
--SET balance = balance - 5000
--WHERE acc_name = 'Member_1';

--SAVEPOINT save_point01;

--UPDATE BankAccounts
--SET balance = balance + 5000
--WHERE acc_name = 'Member_2';

--SAVEPOINT save_point02;

--UPDATE BankAccounts
--SET balance = balance - 3000
--WHERE acc_name = 'Member_4';

SELECT * FROM BankAccounts;
```

Script Output | **Query Result**

Task completed in 0.029 seconds

Save File (Ctrl+S)		
ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	16200
102	Member_2	13000
103	Member_4	12000
105	Member_6	21500

1 row updated.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	16200
102	Member_2	13000
103	Member_4	9000
105	Member_6	21500

Connections

523CS0012_LAB01.sql523CS0012_LAB02.sql523CS0012_LAB03.sql523CS0012_LAB04.sql



0.032 seconds

Worksheet

Query Builder

```
--INSERT INTO BankAccounts (acc_id, acc_name, balance) VALUES (105, 'Member_6', 21500);

--UPDATE BankAccounts
--SET balance = balance - 5000
--WHERE acc_name = 'Member_1';

--SAVEPOINT save_point01;

--UPDATE BankAccounts
--SET balance = balance + 5000
--WHERE acc_name = 'Member_2';

--SAVEPOINT save_point02;

--UPDATE BankAccounts
--SET balance = balance - 3000
--WHERE acc_name = 'Member_4';

--ROLLBACK TO save_point02;

SELECT * FROM BankAccounts;
```

Script Output

Query Result



Task completed in 0.032 seconds

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	16200
102	Member_2	13000
103	Member_4	9000
105	Member_6	21500

Rollback complete.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	16200
102	Member_2	13000
103	Member_4	12000
105	Member_6	21500

Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql



Worksheet | Query Builder

```
--UPDATE BankAccounts
--SET balance = balance - 5000
--WHERE acc_name = 'Member_1';

--SAVEPOINT save_point01;

--UPDATE BankAccounts
--SET balance = balance + 5000
--WHERE acc_name = 'Member_2';

--SAVEPOINT save_point02;

--UPDATE BankAccounts
--SET balance = balance - 3000
--WHERE acc_name = 'Member_4';

--ROLLBACK TO save_point02;
--ROLLBACK TO save_point01;

--COMMIT;

--SELECT * FROM BankAccounts;
```

Script Output x | Query Result x

 Task completed in 0.086 seconds

101	Member_1	22500
104	Member_5	16200
102	Member_2	13000
103	Member_4	12000
105	Member_6	21500

Rollback complete.

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	16200
102	Member_2	8000
103	Member_4	12000
105	Member_6	21500

Commit complete.

Connections

523CS0012_LAB01.sql

523CS0012_LAB02.sql

523CS0012_LAB03.sql

523CS001



0.048 seconds

Worksheet

Query Builder

```
--SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  
--UPDATE BankAccounts  
--SET balance = balance + 8200  
--WHERE acc_name = 'Member_5';  
  
SELECT * FROM BankAccounts;
```

Script Output

Task completed in 0.048 seconds

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	24400
102	Member_2	8000
103	Member_4	12000
105	Member_6	21500

File Edit View Navigate Run Source Team Tools Window Help



...sql 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql x 523CS0012_LAB05.sql x 523CS0012_LAB06.sql x

0.029 seconds

Worksheet Query Builder

```
--UPDATE BankAccounts  
--SET balance = balance + 2000  
--WHERE acc_name = 'Member_5';  
  
SELECT * FROM BankAccounts;
```

Script Output x

Task completed in 0.029 seconds

ACC_ID	ACC_NAME	BALANCE
101	Member_1	22500
104	Member_5	26400
102	Member_2	8000
103	Member_4	12000
105	Member_6	21500

Assignment-4.2 (TCL COMMANDS)

Part A — Setup (DDL + DML)



...ns523CS0012_LAB01.sql523CS0012_LAB02.sql523CS0012_LAB03.sql523CS0012_LAB04.sql523CS0012_LAB05.sql



WorksheetQuery Builder



```
--CREATE TABLE ORDER_S(  
--order_id NUMBER(5) PRIMARY KEY,  
--cus_name VARCHAR2(50),  
--product VARCHAR2(50),  
--amount NUMBER(10)  
--);  
  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartphone', 12000);  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee', 2500);  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop', 28000);  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold Drink', 6500);  
  
SELECT * FROM ORDER_S
```

Script Output xQuery Result x



SQL | All Rows Fetched: 4 in 0.018 seconds

	ORDER_ID	CUS_NAME	PRODUCT	AMOUNT
1	101	Customer_1	Smartphone	12000
2	102	Customer_2	Coffee	2500
3	103	Customer_3	Laptop	28000
4	104	Customer_4	Cold Drink	6500

PART B — COMMIT and ROLLBACK

The screenshot shows a SQL IDE interface with multiple tabs at the top, including '523CS0012_LAB01.sql' through '523CS0012_LAB06.sql'. The 'Query Builder' tab is active, displaying a SQL script. The script includes a table creation statement for 'ORDER_S' with columns 'order_id', 'cus_name', 'product', and 'amount'. It also contains four INSERT statements for different orders and an UPDATE statement to increase the amount of order 102 by 10%. Below the script, a yellow highlight shows the command 'SELECT * FROM ORDER_S'. At the bottom, the 'Query Result' tab displays the output of the SELECT statement as a table with four rows of data.

```
--CREATE TABLE ORDER_S(  
--order_id NUMBER(5) PRIMARY KEY,  
--cus_name VARCHAR2(50),  
--product VARCHAR2(50),  
--amount NUMBER(10)  
--);  
  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartphone', 12000);  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee', 2500);  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop', 28000);  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold Drink', 6500);  
--  
--UPDATE ORDER_S  
--SET amount = amount*1.1  
--WHERE order_id = 102;  
  
--  
SELECT * FROM ORDER_S
```

ORDER_ID	CUS_NAME	PRODUCT	AMOUNT
101	Customer_1	Smartphone	12000
102	Customer_2	Coffee	2750
103	Customer_3	Laptop	28000
104	Customer_4	Cold Drink	6500

...ns523CS0012_LAB01.sql523CS0012_LAB02.sql523CS0012_LAB03.sql523CS0012_LAB04.sql523CS0012_LAB05.sql



0.052 seconds

WorksheetQuery Builder



```
--CREATE TABLE ORDER_S(  
--order_id NUMBER(5) PRIMARY KEY,  
--cus_name VARCHAR2(50),  
--product VARCHAR2(50),  
--amount NUMBER(10)  
--);  
  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartp  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold D  
--  
--UPDATE ORDER_S  
--SET amount = amount*1.1  
--WHERE order_id = 102;  
  
--COMMIT;  
  
--UPDATE ORDER_S  
--SET product = 'Learn'  
--WHERE order_id = 103;  
  
--ROLLBACK;  
--  
SELECT * FROM ORDER_S
```

Script Output xQuery Result x



Task completed in 0.052 seconds

ORDER_ID	CUS_NAME	PRODUCT
101	Customer_1	Smartphone
102	Customer_2	Coffee
103	Customer_3	Learn
104	Customer_4	Cold Drink

...ns523CS0012_LAB01.sql523CS0012_LAB02.sql523CS0012_LAB03.sql523CS0012_LAB04.sql523CS0012_LAB05.sql



0.057 seconds

Worksheet

Query Builder

--CREATE TABLE ORDER_S(
--order_id NUMBER(5) PRIMARY KEY,
--cus_name VARCHAR2(50),
--product VARCHAR2(50),
--amount NUMBER(10)
--);

--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartp
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold D
--
--UPDATE ORDER_S
--SET amount = amount*1.1
--WHERE order_id = 102;

--COMMIT;

--UPDATE ORDER_S
--SET product = 'Learn'
--WHERE order_id = 103;

--ROLLBACK;
--
SELECT * FROM ORDER_S

Script Output x

Query Result x



Task completed in 0.057 seconds

ORDER_ID	CUS_NAME	PRODUCT
101	Customer_1	Smartphone
102	Customer_2	Coffee
103	Customer_3	Laptop
104	Customer_4	Cold Drink

PART C — SAVEPOINT

Connections

523CS0012_LAB01.sql523CS0012_LAB02.sql523CS0012_LAB03.sql523CS0012_LAB04.sql523CS0012_LAB05.sql523CS0012_LAB06

0.068 seconds

Worksheet

Query Builder

--CREATE TABLE ORDER_S(
--order_id NUMBER(5) PRIMARY KEY,
--cus_name VARCHAR2(50),
--product VARCHAR2(50),
--amount NUMBER(10)
--);

--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartphone', 12000);
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee', 2500);
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop', 28000);
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold Drink', 6500);
--
--UPDATE ORDER_S
--SET amount = amount*1.1
--WHERE order_id = 102;

--COMMIT;

--UPDATE ORDER_S
--SET product = 'Learn'
--WHERE order_id = 103;

--ROLLBACK;

SAVEPOINT spl;

--SELECT * FROM ORDER_S

Script Output

Query Result

Task completed in 0.068 seconds

101 Customer_1	Smartphone	12000
102 Customer_2	Coffee	2750
103 Customer_3	Laptop	28000
104 Customer_4	Cold Drink	6500

Savepoint created.

Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql x

Icons: Run, Undo, Redo, Save, Print, Find, Help, etc.

Worksheet Query Builder

```
--CREATE TABLE ORDER_S(  
--order_id NUMBER(5) PRIMARY KEY,  
--cus_name VARCHAR2(50),  
--product VARCHAR2(50),  
--amount NUMBER(10)  
--);  
  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartph  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee'  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop'  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold Dr  
--  
--UPDATE ORDER_S  
--SET amount = amount*1.1  
--WHERE order_id = 102;  
  
--COMMIT;  
  
--UPDATE ORDER_S  
--SET product = 'Learn'  
--WHERE order_id = 103;  
  
--ROLLBACK;  
  
--SAVEPOINT sp1;  
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Compute  
SAVEPOINT sp2;  
  
--SELECT * FROM ORDER_S
```

Script Output x Query Result x

Task completed in 0.051 seconds

Savepoint created.

1 row inserted.

Savepoint created.

Connections

523CS0012_LAB01.sql

523CS0012_LAB02.sql

523CS0012_LAB03.sql

523CS0012_LAB04.sql

0.067 seconds

Worksheet

Query Builder

```

--CREATE TABLE ORDER_S(
--order_id NUMBER(5) PRIMARY KEY,
--cus_name VARCHAR2(50),
--product VARCHAR2(50),
--amount NUMBER(10)
--);

--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartp
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold D
--
--UPDATE ORDER_S
--SET amount = amount*1.1
--WHERE order_id = 102;

--COMMIT;

--UPDATE ORDER_S
--SET product = 'Learn'
--WHERE order_id = 103;

--ROLLBACK;

--SAVEPOINT spl;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Comput
--SAVEPOINT sp2;
--UPDATE ORDER_S
--SET amount = amount + 200
--WHERE order_id = 104;

SELECT * FROM ORDER_S

```

Script Output

Query Result

Task completed in 0.067 seconds

ORDER_ID	CUS_NAME	PRODUCT
101	Customer_1	Smartphone
102	Customer_2	Coffee
103	Customer_3	Laptop
104	Customer_4	Cold Drink
105	Customer_5	Computer System

Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql

0.066 seconds

Worksheet Query Builder

```
--CREATE TABLE ORDER_S(
--order_id NUMBER(5) PRIMARY KEY,
--cus_name VARCHAR2(50),
--product VARCHAR2(50),
--amount NUMBER(10)
--);

--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartp
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold D

--
--UPDATE ORDER_S
--SET amount = amount*1.1
--WHERE order_id = 102;

--COMMIT;

--UPDATE ORDER_S
--SET product = 'Learn'
--WHERE order_id = 103;

--ROLLBACK;

--SAVEPOINT sp1;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Comput
--SAVEPOINT sp2;
--UPDATE ORDER_S
--SET amount = amount + 200
--WHERE order_id = 104;
--ROLLBACK TO sp2;
--ROLLBACK TO sp1;

SELECT * FROM ORDER_S
```

Script Output x Query Result x

Task completed in 0.066 seconds

ORDER_ID	CUS_NAME	PRODUCT
101	Customer_1	Smartphone
102	Customer_2	Coffee
103	Customer_3	Laptop
104	Customer_4	Cold Drink

Part D — SET TRANSACTION

The screenshot displays the SQL Developer interface with a worksheet containing a SQL script. The script includes several INSERT, UPDATE, and COMMIT statements, followed by a ROLLBACK and a SAVEPOINT. The script is executed, and the Script Output pane shows an error message: "ORA-01456: may not perform insert/delete/update operation inside a READ ONLY transaction". The error report indicates that the transaction is in a READ ONLY state, and the action is to commit or rollback the transaction and re-execute the statement.

```
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (101, 'Customer_1', 'Smartphone', 12000);
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (102, 'Customer_2', 'Coffee', 2500);
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (103, 'Customer_3', 'Laptop', 28000);
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (104, 'Customer_4', 'Cold Drink', 6500);
--
--UPDATE ORDER_S
--SET amount = amount*1.1
--WHERE order_id = 102;

--COMMIT;

--UPDATE ORDER_S
--SET product = 'Learn'
--WHERE order_id = 103;

--ROLLBACK;

--SAVEPOINT sp1;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Computer System', 45000);
--SAVEPOINT sp2;
--UPDATE ORDER_S
--SET amount = amount + 200
--WHERE order_id = 104;
--ROLLBACK TO sp2;
--ROLLBACK TO sp1;
--COMMIT;

--SET TRANSACTION READ ONLY;
--SELECT * FROM ORDER_S
UPDATE ORDER_S
```

Script Output x Query Result x Task completed in 0.079 seconds

SET amount = amount + 100
WHERE order_id = 101
Error at Command Line : 38 Column : 8
Error report -
SQL Error: ORA-01456: may not perform insert/delete/update operation inside a READ ONLY transaction

<https://docs.oracle.com/error-help/db/ora-01456/01456.00000> - "may not perform insert, delete, update operation inside a READ ONLY transaction"

*Cause: A non-DDL insert, delete, update or select for update operation was attempted.

*Action: commit (or rollback) transaction, and re-execute

More Details :
<https://docs.oracle.com/error-help/db/ora-01456/>

Connections

523CS0012_LAB01.sql

523CS0012_LAB02.sql

523CS0012_LAB03.sql

523CS0012_LAB04.sql



0.089 seconds

Worksheet

Query Builder

```
--UPDATE ORDER_S
--SET product = 'Learn'
--WHERE order_id = 103;

--ROLLBACK;

--SAVEPOINT sp1;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Comput
--SAVEPOINT sp2;
--UPDATE ORDER_S
--SET amount = amount + 200
--WHERE order_id = 104;
--ROLLBACK TO sp2;
--ROLLBACK TO sp1;
--COMMIT;

--SET TRANSACTION READ ONLY;
--SELECT * FROM ORDER_S
--UPDATE ORDER_S
--SET amount = amount + 100
--WHERE order_id = 101;
--COMMIT;

--SET TRANSACTION READ WRITE;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Comput
--UPDATE ORDER_S
--SET amount = amount + 100
--WHERE order_id = 101;

SELECT * FROM ORDER_S
```

Script Output

Task completed in 0.089 seconds

1 row inserted.

1 row updated.

ORDER_ID	CUS_NAME	PRODUCT
101	Customer_1	Smartphone
102	Customer_2	Coffee
103	Customer_3	Laptop
104	Customer_4	Cold Drink
105	Customer_5	Computer System

Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_LAB04.sql

0.109 seconds

Worksheet Query Builder

```
--WHERE order_id = 103;

--ROLLBACK;

--SAVEPOINT sp1;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Comput
--SAVEPOINT sp2;
--UPDATE ORDER_S
--SET amount = amount + 200
--WHERE order_id = 104;
--ROLLBACK TO sp2;
--ROLLBACK TO sp1;
--COMMIT;

--SET TRANSACTION READ ONLY;
--SELECT * FROM ORDER_S
--UPDATE ORDER_S
--SET amount = amount + 100
--WHERE order_id = 101;
--COMMIT;

--SET TRANSACTION READ WRITE;
--INSERT INTO ORDER_S (order_id, cus_name,product, amount) VALUES (105, 'Customer_5', 'Comput
--UPDATE ORDER_S
--SET amount = amount + 100
--WHERE order_id = 101;
--ROLLBACK;

SELECT * FROM ORDER_S
```

Script Output x

Task completed in 0.109 seconds

104 Customer_4	Cold Drink
105 Customer_5	Computer System
Rollback complete.	
ORDER_ID	CUS_NAME
PRODUCT	
101 Customer_1	Smartphone
102 Customer_2	Coffee
103 Customer_3	Laptop
104 Customer_4	Cold Drink

READ COMMITTED

The screenshot shows the SQL Developer interface with a query executed. The query is: `--SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT * FROM ORDER_S`. The result is displayed in the Script Output window, showing a table with 4 columns: ORDER_ID, CUS_NAME, PRODUCT, and AMOUNT. The data is as follows:

ORDER_ID	CUS_NAME	PRODUCT	AMOUNT
101	Customer_1	Smartphone	12000
102	Customer_2	Coffee	2750
103	Customer_3	Laptop	28000
104	Customer_4	Cold Drink	6500

Connections x 523CS0012_LAB01.sql x 523CS0012_LAB02.sql x 523CS0012_LAB03.sql x 523CS0012_L

0.054 seconds

Worksheet Query Builder

```
--UPDATE ORDER_S
--SET amount = amount + 2500
--WHERE order_id = 102;
COMMIT;
```

Script Output x

Task completed in 0.054 seconds

1 row updated.

Commit complete.

SERIALIZABLE

The screenshot shows a SQL IDE interface. The top toolbar includes icons for running queries, saving, and other database operations. The main window is titled 'Worksheet' and contains the following SQL code:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;  
  
UPDATE ORDER_S  
SET amount = amount + 100  
WHERE order_id = 101;  
SELECT * FROM ORDER_S
```

Below the code editor, the 'Query Result' tab is active, displaying the results of the SELECT statement. The status bar indicates 'All Rows Fetched: 4 in 0.001 seconds'.

ORDER_ID	CUS_NAME	PRODUCT	AMOUNT
1	101 Customer_1	Smartphone	12100
2	102 Customer_2	Coffee	10250
3	103 Customer_3	Laptop	28000
4	104 Customer_4	Cold Drink	7500

Connections

523CS0012_LAB01.sql523CS0012_LAB02.sql523CS0012_LAB03.sql523CS0012_LAB04.sql



Worksheet

Query Builder

UPDATE ORDER_S

SET amount = amount + 100

WHERE order_id = 101;

Script Output



ScriptRunner Task



